AI 开发必修 · 2026

# AI 开发必知的五类协议

从「双方约定」出发，逐层拆开 TCP / HTTP / WebSocket / SSE / JSON-RPC / gRPC / MCP 的真实关系，并修正原图中的层次错位与关键缺失。

01 **协议 = 双方的约定**
语法、语义、时序，三者缺一不可。

02 **协议是叠加，不是并列**
gRPC = Protobuf + HTTP/2 + 调用约定。

03 **HTTP 与 WebSocket**
握手、帧、心跳、重连，逐项对齐。

04 **SSE 与 JSON-RPC / gRPC**
流式传输方式 vs 调用约定标准。

05 **MCP 与全景收束**
三原语、一对一连接、承载方式，最后一张勘误表收尾。

**为什么会有这份修订版**

原图五张卡片覆盖了协议栈的骨架，但有 **3 处层次错位** 和 **3 处关键机制缺失**。照原图理解，写代码时会在「为什么 SSE 和 RAG 并列」「MCP 到底能不能跑在云上」这类地方绕弯路。最后一章的勘误表是完整修改清单。

PART 01 · 协议基础

# 协议 = 语法 + 语义 + 时序

协议不是「一句话约定」，而是一份**双方都要实现**的规则集。少对齐任何一项，通信都会出问题。

📐

### 语法 · Syntax

消息长什么样：字段名、类型、编码方式、字节序。

```
{"jsonrpc": "2.0",
 "method": "search",
 "id": 1}
```

🧠

### 语义 · Semantics

每个取值代表什么：状态码含义、错误码归类、字段的约束条件。

```
-32601 = 方法不存在
-32602 = 参数非法
200 + errBody = 语义错位
```

⏱️

### 时序 · Timing

谁先谁后：请求顺序、超时时长、重试次数、并发与背压。

```
connect → handshake
→ request → stream
timeout 30s / retry ×2
```

最容易踩的坑

**语法对齐 ≠ 通信正确。**两边字段名一模一样，但错误码含义不同、或者 200 里面装着错误内容——这类「静默失败」比直接报错难查十倍。对接时先对齐语义表，再对齐字段表。

PART 01 · 分层关系

# 它们是叠加承载，不是三层并列

原图把 JSON-RPC/gRPC、HTTP/WebSocket、TCP 画成并列的三层，实际关系是**上层建立在下层之上**。

### 应用语义层　调什么方法、传什么参数

**🔧 JSON-RPC 2.0**
纯调用约定，不绑定传输 · method / params / id

**🧬 gRPC**
调用约定 + 序列化 + 传输三合一 · .proto → stub

**🌐 REST / OpenAPI**
资源导向，契约靠描述文件 · GET / POST / 204

**🧰 MCP**
面向 AI 的调用约定 · tools / resources / prompts

▼ ▼ ▼

### 序列化层　消息如何变成字节

**📦 JSON**
人可读、自描述，体积大

**📐 Protocol Buffers**
schema 驱动，二进制紧凑

**✍️ Text / SSE 帧**
行分隔的文本事件流

▼ ▼ ▼

### 传输层　字节如何在网络上流动

**🚌 HTTP / HTTP2**
请求响应语义 + 头部压缩 + 多路复用

**🔌 WebSocket**
HTTP 升级后转独立帧协议，全双工

**📡 SSE**
HTTP 响应体内持续推事件，单向

▼ ▼ ▼

### 网络层　字节如何抵达对端

**🧱 TCP**
可靠有序字节流，面向连接；WebSocket / HTTP / SSE 最终都跑在它上面

**💨 UDP**
无连接、不保证到达；QUIC 在其上补齐可靠性与多路复用

**关键修正**

**gRPC 不是独立的一层**：它 = Protobuf 序列化 + HTTP/2 传输 + gRPC 调用约定。**JSON-RPC 是传输无关的规范**，实际多跑在 HTTP 或 WebSocket 上。**WebSocket 也不是应用层协议**——它是 HTTP 升级而来的传输层协议。

PART 02 · 传输层

# HTTP 与 WebSocket：一次性 vs 持续性

## 📨 HTTP / HTTP2

请求 → 响应，一问一答

- **无状态**：会话靠 Header / Cookie 维持，不在连接里
- **连接复用**：HTTP/1.1 keep-alive；HTTP/2 多路复用 + HPACK 头部压缩 + 二进制帧
- **流式响应**：HTTP/1.1 用 chunked 编码，HTTP/2 用 DATA 帧
- **服务端推送已弃用**（HTTP/2 Server Push），推流改用 SSE 或 WebSocket
- 生态成熟：网关、缓存、鉴权、限流全都现成

## 🔗 WebSocket

握手后转独立帧协议，全双工

- **握手**：`GET /ws + Upgrade: websocket + Sec-WebSocket-Key` → **101 Switching Protocols**
- **帧结构**：FIN / RSV / opcode（text·binary·close·ping·pong）/ mask / payload length——不再是 HTTP 报文
- **心跳**：ping / pong 帧探测半开连接，穿越 NAT 与代理超时的必备机制
- **断线重连**：协议不提供，须自行实现指数退避 + 会话续传
- 典型场景：IM、协同编辑、实时语音、行情推送

**怎么选**

只要「服务端单方面推给客户端」，用 **SSE** 更轻——它就是普通 HTTP，不需要额外握手和心跳；需要**双向实时**（客户端也要高频上行）才上 WebSocket。HTTP 与 WebSocket 也能共存于同一服务，靠路径区分。

PART 02 · 流式传输

# SSE：把单向流塞进普通 HTTP

## 📡 机制

Content-Type: text/event-stream

- **服务端不关闭响应体**，持续追加事件帧；每帧以空行结束
- 可选字段：`data` / `event` / `id` / `retry`
- 浏览器原生 **EventSource**：自动重连，并用 **Last-Event-ID** 续传，服务端可据此补发
- 局限：**单向**（上行需另开 HTTP）；HTTP/1.1 下同域并发连接约 6 条，需关代理缓冲（`X-Accel-Buffering: no`）

```
HTTP/1.1 200 OK
Content-Type: text/event-stream

data: {"type":"token","v":"你"}

data: {"type":"done"}
```

## 🔍 层次澄清（重要）

原图把 SSE 与「向量化 / 检索」画成了同一层的流水线

应用模式

**RAG（向量化 / 检索 / 重排）**发生在**服务端内部**，是一次检索 + 一次生成的组合算法。

传输方式

**SSE**只负责「把服务端产出的内容增量推给客户端」，它不关心内容从哪来。

正确关系

RAG 在服务端算完 → 结果通过 SSE 推给前端。二者是**上下层**，不是并列步骤。

工程现实

OpenAI / Anthropic 的流式 chat completion 接口用的就是 SSE；工具调用回合同样是 SSE 上的一段 JSON。

**对比**

轮询：反复建请求，浪费且延迟高；长轮询：服务端挂起连接，仍需自己重连；**SSE：纯 HTTP 语义 + 浏览器自带重连**，是 AI 流式输出的事实标准。

PART 02 · 调用约定

# JSON-RPC / gRPC：让接口不再靠口口相传

## 🧬 JSON-RPC 2.0

规范本身与传输无关，最常跑在 HTTP 或 WebSocket 上

```
// 请求
{"jsonrpc": "2.0", "method": "search_docs",
 "params": {"q": "向量检索"}, "id": 1}

// 成功响应
{"jsonrpc": "2.0", "result": {"hits": 3},
"id": 1}

// 错误响应
{"jsonrpc": "2.0",
 "error": {"code": -32601, "message":
"Method not found"},
 "id": 1}
```

- **id** 关联请求与响应；**无 id 即 notification**，服务端不回复
- **批量请求**：传 JSON 数组，服务端返回结果数组
- 保留错误码：-32700 解析失败 / -32600 请求无效 / -32601 方法不存在 / -32602 参数非法 / -32603 内部错误

## 🧱 gRPC

schema 先行，双端生成桩代码

- **.proto** 定义 service / method / 消息类型 → **protoc** 生成各语言客户端与服务端代码，签名不可能漂移
- Protobuf 二进制编码，体积小、解析快
- 跑在 **HTTP/2** 上，四种调用模式：**unary / server-streaming / client-streaming / bidirectional**
- 代价：浏览器原生不支持，需 gRPC-Web 或代理；调试不如 JSON 直观

**措辞修正**

原图说「像调本地一样」——准确说法是**「像调函数一样」**。它仍是远程调用：有序列化开销、有网络延迟、可能部分失败。因此必须配套**超时、幂等键、重试与熔断**。

**和 MCP 的关系**

**MCP 用的正是 JSON-RPC 2.0**，只是换了承载方式（stdio / Streamable HTTP）。这两张原图之间的暗线就接在这里。

PART 03 · AI 应用层

# MCP：把「AI 接外部世界」标准化

Anthropic 提出并开源，现由 Linux Foundation 旗下 **Agentic AI Foundation**（Anthropic、Block、OpenAI 共同发起）托管，走中立治理。

## 🔧 Tools

模型可调用的**动作**，通常有副作用（写库、发消息、扣款）。执行前一般需用户确认。

`search · create · execute`

### Resources

可读取的**上下文数据**，通常只读，按 URI 寻址。模型据此「知道」有什么可查。

`file:// · db://schema`

### Prompts

用户**显式触发**的模板化工作流，不由模型自行决定使用。

`/code-review · /summarize`

### 架构：Host / Client / Server

Host 为每个 Server 建立一个专属 Client，严格 1 : 1

- **Host**：Claude Code、Claude Desktop、IDE 等 AI 应用，持有对话与权限
- **Client**：Host 内部的协议实现，一个 Client 只服务一个 Server
- **Server**：暴露 tools / resources / prompts 的独立程序
- **隐私边界**：Server 看不到完整对话，只看到本次请求——权限最小化的关键

### 承载与定位

JSON-RPC 2.0 消息，两种标准传输

- **stdio**：本地子进程，读写标准输入输出，本地工具零端口暴露
- **Streamable HTTP**：单一端点 POST，响应可为一个 JSON 对象或请求级 SSE 流
- **修正**：MCP **不等于「云服务接口」**——它与「云」无关，本地进程同样是 Server，stdio 才是本地场景的主力
- 价值在于**标准化与可发现**：同一份 Server 换个 Host 就能用

PART 03 · 全景收束

## 一张图串起来，外加选型结论

| 语义 越往上越贴近「语义」 | |
|---|---|
| JSON-RPC | gRPC |
| MCP | REST |

▼ ▼ ▼

| 序列化 | | |
|---|---|---|
| JSON | Protobuf | SSE 帧 |

▼ ▼ ▼

| 传输 | | |
|---|---|---|
| **HTTP / 2** | **WebSocket** | **SSE** |

▼ ▼ ▼

| 网络 | |
|---|---|
| **TCP** | **UDP / QUIC** |

## ✅ 按需求选型

普通业务接口

**REST + JSON / HTTP** · 生态最全，别过度设计

内部服务、强 schema、跨语言

**gRPC** · 性能与契约强，浏览器需 gRPC-Web

AI 调用自己的后端能力

**MCP** · 换 Host 不用重写，换业务不用改 Agent

模型流式吐字

**SSE** · 事实上标准，自带重连与续传

双向实时（IM / 协同）

**WebSocket** · 心跳与重连要自己实现

对外开放第三方集成

**REST** · MCP 是给 AI 用的，不是给人用的

**一句话**

协议选型不是选「最强」的，是选**出问题最容易定位**的。层次清晰，排查才有落点。

PART 03 · 勘误

# 原图 → 修订：六处需要改的地方

## 图 1 三层并列

**原图**

~~JSON-RPC / gRPC、HTTP / WebSocket、TCP 三层并列~~

**问题**

误当作同级替代关系

**修订**

叠加承载：**gRPC = Protobuf + HTTP/2 + gRPC 约定**；JSON-RPC 传输无关，多跑 HTTP / WebSocket；WebSocket 是传输层协议

## 图 1 协议定义过宽

**原图**

~~「协议就是双方遵守同一套规则」~~

**问题**

定义过宽，没给拆解

**修订**

拆成**语法（长什么样）、语义（什么意思）、时序（谁先谁后）**三要素，缺一即无法互通

## 图 2 WebSocket 缺关键机制

**原图**

~~WebSocket = 全双工 · 实时推送~~

**问题**

缺关键机制

**修订**

补：**101 升级握手、帧结构、ping/pong 心跳、断线重连与指数退避**；并指出 SSE 更适合单向推流

### 图 3 措辞低估远程调用

**原图**

~~「像调本地一样」~~

**问题**

忽略远程调用特性

**修订**

应为**「像调函数一样」**：仍有序列化开销、网络延迟与部分失败，必须配超时、幂等键、重试与熔断

### 图 4 层次错位

**原图**

~~SSE 与「向量化 / 检索」并列为流水线~~

**问题**

层次错位

**修订**

**SSE 是传输方式，RAG 是应用模式**：检索在服务端内部完成，产出结果再经 SSE 推送，二者上下层

### 图 5 MCP 定位过窄、原语不全

**原图**

~~MCP = AI 时代的云服务接口；只讲 tool~~

**问题**

定位过窄、原语不全

**修订**

MCP 与「云」无关，**stdio 本地进程同样是 Server**；原语应为 **Tools / Resources / Prompts** 三类；补 Host-Client-Server 一对一架构与两种传输

## 收束 · 记住这三句

**① 协议 = 语法 + 语义 + 时序**

字段对齐只是第一步，语义一致才是通信正确的前提。

**② 协议是叠加的，不是并列的**

gRPC 跑在 HTTP/2 上，JSON-RPC 跑在 HTTP/WS 上，MCP 跑在 JSON-RPC 上。一层层往上叠。

**③ 选型看需求，不看强弱**

单向推流 SSE，双向实时 WebSocket，AI 接工具 MCP，通用业务 REST。

协议是约定，分层是地图，出错时才知道该往哪一层找。剩下的问题，随时来问。

AI 知识导师 · 协议知识地图修订版